Smart Agriculture Recommendation System: An AI-Powered Backend API

I'm excited to share a comprehensive overview of my recent project: The **Smart Agriculture Recommendation API**.

This system integrates multiple machine learning models and exposes them through a highperformance RESTful API built using FastAPI. It’s designed to assist farmers and agricultural analysts in making data-driven decisions ranging from crop selection to plant disease detection.

Here is a deep dive into the architecture, the machine learning models, and the engineering challenges overcome in this project! ᵛ

Project Overview & Core Features

The API acts as the core predictive engine for four distinct agricultural modules:

1 **Crop Recommendation System**
Predicts the most suitable crop to grow given soil metrics (Nitrogen, Phosphorus, Potassium, pH) and environmental factors (Temperature, Humidity, Rainfall).

2 **Fertilizer Recommendation System**
Evaluates nutrient deficiencies in the soil and considers the crop type to recommend the most optimal fertilizer.

3 **Crop Yield Prediction**
Estimates expected crop production (yield per hectare) based on regional parameters, rainfall, pesticide use, and temperature.

4 **Plant Disease Detection System**
An image-based deep learning module that processes uploaded plant leaf images and classifies them into 14 different disease or healthy categories.

Technology Stack & Engineering Practices

The project demonstrates a seamless integration of Data Science and Backend Engineering:

- **Backend Framework**: FastAPI (for asynchronous HTTP request handling & automated Swagger docs)
- **Machine Learning**: Scikit-Learn (Random Forest algorithms)
- **Deep Learning (Computer Vision)**: PyTorch & Torchvision (Custom CNN architecture)
- **Data Processing & Vision**: Pandas, NumPy, OpenCV, Pillow
- **Model Serialization**: j obl i b for ML models, PyTorch state dictionaries for the CNN • **Deployment**: Docker & Uvicorn

---

The Machine Learning Models

1. Crop Recommendation & Yield Prediction (Tabular Data)

- **Algorithm**: RandomFor es t Cl as s i f i er (for crop selection) and RandomFor es t Regr es s or (for crop yield).
- **Input Handling**: Continuous environmental features combined with LabelEncoded categorical variables (like geographic area and crop type).
- **Performance**: Strong resilience to non-linear relationships in environmental data.

2. Plant Disease Detection (Image Data)

- **Architecture**: Custom PyTorch Convolutional Neural Network (CNN).
- **Training Specs**: Images resized to 128x128. Trained to achieve an impressive **~ 94% validation accuracy** across 14 distinct leaf categories (including Apple, Corn, Potato, and Tomato diseases).

Overcoming Engineering Challenges: External Model Hosting

The Challenge: The custom PyTorch CNN weights file (di s eas e_model . pt h) exceeded standard GitHub and free-tier cloud hosting size limits.

The Solution: I decoupled the large models from the repository. The trained models are now securely hosted externally on the **Hugging Face Model Hub**.

I engineered the backend application to **dynamically download** these required models directly into the internal cache precisely upon server startup or during lazy-loading inferences. This keeps the version control footprint lightweight while preserving the application's robust predictive accuracy in production!

API Endpoint Highlights

The system exposes clean, documented POST endpoints:

- POST / c r op/ r ec ommendat i on - J SON payload of soil metrics -> Suggested Crop.
- POST / yi el d/ pr edi c t i on - J SON payload of climate stats -> Estimated Yield.
- POST / di s eas e/ det ec t i on - mul t i par t / f or m- dat a image upload -> Disease Classification & Confidence Score.

Moving Forward

The foundation is built for production scalability. My next focus areas include:

- Developing a modern, mobile-friendly Web UI.
- Integrating real-time edge detection directly from mobile cameras.
- Integrating external Weather APIs for automated parameter fetching.
- Exploring Transfer Learning (e.g., ResNet) to further optimize the computer vision models.